Updated Acceptance Criteria

User Story ID and Name	AC ID	Description of Acceptance Criteria	Status (completed, toDo, inProgress)
1.Choose board type and size	1.1	**AC 1.1 <Select board type>** Given I am on the main menu When I select option English, Hexagon, or Diamond Then the selected board type should be stored	complete
	1.2	**AC 1.2 <Select board size>** Given I am on the main menu When I input a board size And board size is within valid range for its type Then the inputted size should be stored	complete
	1.3	**AC 1.3 <Reject invalid board sizes>** Given I am on the main menu And I've inputted a board size out of acceptable range When I try to start a new game Then I receive an error message And no board is generated	complete
	1.4	**AC 1.4 <Default configuration when opening program>** When I boot the program Then the board size and type are set to default configurations	complete
2. Beginning a game	2.1	**AC 2.1 <Beginning a game with default board configuration>** Given I am on the main menu And I am on default board configuration When I start a new game Then the game board should be displayed with default configuration And all board positions should contain pegs except the center position And the center position should be empty And the game state should be "in progress"	complete

	2.2	**AC 2.2 <Beginning a game applies the selected board configuration>** Given I am on the main menu And I have selected a specific board type and size When I start a new game Then the board should be displayed And the board should match the selected type And the board should match the selected size And all board positions should contain pegs except the center position And the center position should be empty And the game state should be "in progress"	com plete
3. View the Current Board State	3.1	**AC 3.1 <Display the current state of the board after a successful move>** Given the game is in progress When I make a valid move Then the GUI should update the board to show the new game state	com plete
4. Make a Move	4.1	**AC 4.1 <Make a Move>** Given the game is in progress And a peg is located at a valid board position And an adjacent position contains another peg And the position two spaces away in the same direction is empty When the player selects the peg and performs a jump to the empty position Then the peg moves to the empty position And the jumped peg is removed from the board And the original position becomes empty And the board state is updated	com plete
	4.2	**AC 4.2 <Reject Selection of Empty Position>** Given the game is in progress When the player attempts to select a position with no peg Then the selection does not occur	com plete
	4.3	**AC 4.3 <Reject moves that jump over an empty position>** Given the game is in progress And a peg is selected	com plete

		And the adjacent position in the selected direction is empty When the player attempts to make a move Then the move is rejected And the game state remains unchanged	
	4.4	**AC 4.4 <Reject moves that land in an occupied position>** Given the game is in progress And a peg is selected And the landing position is occupied When the player attempts to make a move Then the move is rejected And the game state remains unchanged	com plete
	4.5	**AC 4.5 <Reject moves that jump more than one peg>** Given the game is in progress And a peg is selected And there is more than one peg between the start and landing positions When the player attempts to make a move Then the move is rejected And the game state remains unchanged	com plete
	4.6	**AC 4.6 <Reject moves that put a peg outside the board boundaries>** Given the game is in progress And the landing position is outside the board When the player attempts to make a move Then the move is rejected And the game state remains unchanged	com plete
	4.7	**AC 4.7 <Reject moves in non-allowed directions>** Given the game is in progress And the selected move is not orthogonal or diagonal When the player attempts to make a move Then the move is rejected And the game state remains unchanged	com plete
	4.8	**AC 4.8 <Only process one move at a time>** Given the game is in progress When the player completes a valid move	com plete

		Then no additional moves are processed until the current move is finished	
	4.9	**AC 4.9 <Reject moves to neighboring positions>** Given the game is in progress And a peg is selected And the landing position is neighbors to the selected position When the player attempts to make a move Then the move is rejected And the game state remains unchanged	com plete
5. Detect Game Over	5.1	**AC 5.1 <Detect when no more valid moves>** Given the game is in progress When the player has made a valid move And there are no longer any valid moves available Then the game should end And the player can no longer make moves	com plete
	5.2	**AC 5.2 <Detect bugged start>** When a new game begins And, somehow, there are no valid moves to make Then the game should end And the player can no longer make moves	canc elled
6. Determine Performance	6.1	**AC 6.1 <Rank based on remaining pegs>** Given the game is in progress When the game ends Then the player should receive a ranking based on the number of pegs remaining	toDo
7. Reset	7.1	**AC 7.1 <Reset during game>** Given the game is in progress When the player chooses to reset Then the board should return to its original state (all pegs filled except the center) And the game state should reset too	com plete
	7.2	**AC 7.2 <Reset after game>** Given the game has ended When the player chooses to reset	com plete

		Then the board should return to its original state (all pegs filled except the center) And the game state should reset too	
8. Reconfigure Board	8.1	**AC 8.1 <Reconfigure during the game>** Given the game is in progress When the player chooses to start a new game Then we delete the current game state and board And return to the main menu, where the player can choose board size and type	com plete
	8.2	**AC 8.2 <Reconfigure after the game>** Given the game is over When the player chooses to start a new game Then we delete the current game state and board And return to the main menu, where the player can choose board size and type	com plete
9. Automated moves	9.1	**AC 9.1 <Make a random move>** Given the game is ongoing When the player presses the autoplay button Then the board makes a random valid move	com plete
	9.2	**AC 9.2 <Don't make automated move if game over>** Given the game is over When the player presses the autoplay button Then there should be no change to game state	com plete
10. Randomize board	10.1	**AC 10.1 <Randomize board>** When the player presses the randomize button Then every valid space should randomly become either an empty square or a square with a peg	com plete
	10.2	**AC 10.2 <Don't randomize invalid cells>** When the player presses the randomize button Then all cells which are invalid should remain invalid	com plete
	10.3	**AC 10.3 <Randomized board must be valid>** When the player presses the randomize button Then the randomized board should be one where valid move(s) are available	com plete

Summary of Source Code

Source code file name	Production code or test code?	# lines of code
AutomatedGame.java	Production code	30
GameSession.java	Production code	86
ManualGame.java	Production code	58
Board.java	Production code	233
CellState.java	Production code	6
DiamondBoard.java	Production Code	34
EnglishBoard.java	Production Code	42
HexagonBoard.java	Production Code	83
Move.java	Production Code	44
Square.java	Production Code	45
BoardView.java	Production Code	219
CellStateTest.java	Test Code	10
DiamondBoardTest.java	Test Code	39
EnglishBoardTest.java	Test Code	298
HexagonBoardTest.java	Test Code	73
SquareTest.java	Test Code	42
	Total	1,342

Production Code vs. User Acceptance Criteria

User Stories

User Story ID	User Story

1	Choose board type and size
2	Beginning a game
3	View the current board state
4	Make a move
5	Detect game over
6	Determine performance
7	Reset
8	Reconfigure board
9	Automated moves
10	Randomize board

User Story ID and Name	**AC ID**	**Class Name(s)**	**Method Name(s)**	**Status (complete or not)**	**Notes**
1: Choose board type and size	1.1	BoardView	listener for radio button group within start() method	Complete	
	1.2	BoardView	setOnAction for comboBox within start() method	Compete	
	1.3	BoardView	-setOnAction for comboBox within start() method	Complete	ComboBox limits size choices to valid ones
	1.4	BoardView	BoardView class definition, drawBoard	Complete	When we first created BoardView, we create a

					new board with default configuration s and display it.
2: Beginning a game	2.1	See 1.4	See 1.4	Complete	See 1.4
	2.2	--	--	Cancelled	No longer necessary, as there is no main menu
3: View the current board state	3.1	BoardView	canvas.setOnMouseClicked, autoplayButton.setOnAction, drawBoard	Complete	Mouse clicks and the autoplay button both redraw the board after a move is made
4: Make a move	4.1	BoardView, ManualGame, AutomatedGame, Board	canvas.setOnMouseClicked, autoplayButton.setOnAction, handleClick, makeRandomMove, makeMove	Complete	
	4.2	See 4.1	See 4.1	Complete	
	4.3			Complete	
	4.4			Complete	
	4.5			Complete	
	4.6			Complete	
	4.7			Complete	
	4.8			Complete	
	4.9			Complete	

5: Detect game over	5.1	Board	validMovesExist	Complete	
	5.2	--	--	Cancelled	Note: There is no reason for the game to ever initialize without a valid move being possible
6: Determine performance				Not complete	
7: Reset	7.1	BoardView, GameSession, Board	resetButton.setOnAction, handleResetButton resetBoard	Complete	
	7.2	7.1	See 7.1		
8: Reconfigure board	See user story 1	See user story 1	See user story 1	Complete	See user story 1
9: Automated moves	9.1	BoardView, automatedController, Board	autoplayButton.setOnAction, makeRandomMove, getRandomMoves, validMovesExist	Complete	
	9.2	BoardView, automatedController, Board	makeRandomMove, getRandomMoves, validMovesExist	Complete	
10: Randomize	10.1	BoardView, Board	randomButton.setOnActio	Complete	

board			n, randomize		
	10.2	BoardView, Board	randomButton.setOnAction, randomize	Complete	
	10.3	BoardView, Board	randomButton.setOnAction, randomize	Complete	

Tests vs. User Stories/Acceptance Criteria

Automated tests directly corresponding to the acceptance criteria of the above user stories

User Story ID and Name	**Acceptance Criterion ID**	**Class Name (s) of the Test Code**	**Method Name(s) of the Test Code**	**Status (Pass/Fail/In Progress)**
1 -- Choose board type and size				
2 -- Beginning a game	2.1	EnglishBoardTest	testBoardSize testSizeSevenBoard	Pass
...	2.2	HexagonBoardTest, DiamondBoardTest	testHexBoardSize testSizeSevenHexBoard testSizeFifteenHexBoard testDiBoardSize testSizeSevenDiBoard testSizeFifteenDiBoard	Pass
3 -- View the current board state	3.1	SquareTest, CellStateTest	testInitializePeg testInitializeEmpty testInitializeInvalid testSetState testRow testColumn	Pass

			testEqualsThreeStates	
4 -- Make a move	4.1	EnglishBoardTest	testValidOrthogonal testValidDiagonal testValidDiagonal2 testJumpRemovesPeg testDiagJumpRemovesPeg	Pass
...	4.2	EnglishBoardTest	testInvalidStart testEmptyStart	Pass
...	4.3	EnglishBoardTest	testEmptyJump testEmptyDiagJump	Pass
...	4.4	EnglishBoardTest	testOccupiedLanding	Pass
...	4.5	EnglishBoardTest	testDoubleHop testLongMove	Pass
...	4.6	EnglishBoardTest	testInvalidLanding testOutOfBounds	Pass
...	4.9	EnglishBoardTest	testNeighborMove	Pass
5 -- Detect game over	5.1	EnglishBoardTest	testIfNoValidMoves testDetectValidMoves	Pass
6 -- Determine performance				
7 -- Reset	7.1, 7,2	EnglishBoardTest	resetBoard	Pass
8 -- Reconfigure board				

9 -- Automated Moves				
10 -- Randomize Board	10.2	EnglishBoardTest	testDontRandomizeInvalid	Pass

Manual tests directly corresponding to some acceptance criteria

User Story ID	Acceptance Criterion ID	Test Case Input	Test Oracle (Expected Output)	Notes
1	1.1-1.3	See test inputs for user story 8	See test oracles for user story 8	I ultimately nixed the “main menu” concept because it was simpler and more usable to simply have a combo box for size and a set of radio buttons for type which update the board in real time as they are modified.
	1.4	Starting a new game	Board initializes as a size 7 English board	Success
8	8.1	Changing size or board type during an active game	Board resets itself, updating its configuration.	Success
	8.2	Changing size or board type when a game is over	Board resets itself, updating its configuration	
9	9.1	Press autoplay button when game is not over	A random valid move is made	Repeated trials showed random results. There were no occasions where it failed to make

				a move (unless the game was over) and no moves made that were invalid.
	9.2	Press autoplay button when game is over	No move is made	Success
10	10.1	Pressed randomize button	All cells are randomly empty or pegged	Repeated trials showed random results. Resetting the game did *not* cause the same configurations to appear, so it was truly random as desired.
	10.3	Modified code so only one in fifty squares had a peg in them, then pressed randomized button	A randomized board where at least one valid move is possible.	The way I guaranteed the board will have valid move(s) was by having the randomize function loop until it creates a board with one or more valid moves available. When only one square in fifty has a peg, the chance of invalid boards being created is much higher, so the 100% valid boards I saw confirm that 10.3 is implemented.

Class hierarchy. In order to implement a class hierarchy for manual and automated games, I renamed my gameController class to ManualGame and created a new class called Game from which it extends. Then, of course, I created AutomatedGame and also had that extend from Game. The common features which Game holds are the board and gameOver fields. Board is the Board object which both game types reference and gameOver is a variable tracking whether or not a game has ended. Game has a method called handleResetButton which resets the board and sets gameOver to false, which both ManualGame and AutomatedGame inherit. They also both inherit getters and setters for board and gameOver. ManualGame inherits the fields of Game and also has the fields selectedRow and selectedCol to keep track of which cell the user has clicked, and it has a method called handleClick which the BoardView invokes upon each mouse click. It also has getters and setters for selectedRow and selectedCol. AutomatedGame does not have any fields besides the ones which it inherits from Game. It has a function called makeRandomMove which is analogous to handleClick, but it is activated by a button press instead of a mouse click. In retrospect, I suppose I could have created an abstract method in Game called "moveEvent," or something like that, which both of these classes could have extended. However, I think the more descriptive names of each class are also valuable. Another difference between ManualGame and AutomatedGame is that AutomatedGame has a unique method called getValidMoves which it uses to create a list of all possible valid moves so that it can randomly select one of them to perform. This function is somewhat similar to the validMovesExist method from the Board object in that it exhaustively checks every move for every remaining peg on the board to determine what options are available, although it differs in that instead of merely returning a boolean it returns a full list. Rather than implement two "game modes," I placed the invocation functions for both classes' move event methods side-by-side in BoardView, with handleClick being triggered by mouse clicks and makeRandomMove being triggered by the press of an autoplay button. I thought it was easier to implement (and a more interesting game) if you could choose at any time to either make a move yourself or have the board make a move randomly. This allows you to challenge yourself by choosing to trigger a random move partway through a game, forcing yourself to make the best of whatever it does to your position. ***UPDATE:*** After Claude's refactoring, the Game class has been removed entirely. ManualGame and AutomatedGame no longer extend anything. Now there is a GameSession object which is *not* a parent of ManualGame or AutomatedGame, but which *has* both of them and which rules over them with an iron fist. This technically breaks the class hierarchy requirement, but I hope it is clear that I made the attempt to implement it as such and that I only changed it because it was clear that it was not the most effective implementation for this project.

LLM Feedback: Getting Claude's feedback really helped me to understand the design principles discussed in class because now they were being applied to a code base that I was intimately familiar with. In particular, my understanding of what cohesion is has greatly improved; it is simply the tendency I already had to place related functions in the same classes, and its violation was simply that odd feeling I'd have of, "Oh, how strange I put it in *that* class when it feels like something which would be in *this*" class. Its main feedback was that I was violating cohesion by having a "getValidMoves" function in AutomatedGame. Although AutomatedGame is the only class that uses it (so far), Claude made a good point that getValidMoves is *very* similar to many of the functions that were defined for Board, and so it should go in there to improve cohesion. I was able to fix that issue by myself without any help from Claude. The largest design issue — and the one that caused the most bugs for me while I was coding this — was the relationship

between BoardView, AutomatedGame, and ManualGame. For starters, BoardView was keeping track of a lot of things and handling a lot of things, which is not what a View module is supposed to do. Secondly, I had multiple subjective realities going on: BoardView, ManualGame, and AutomatedGame all had their own gameOver variables, and if I wanted to make a modification to gameOver at any point I had to make sure to do it for all three realities so no strange behavior would occur. Claude helped me understand modularity better by pointing out that most of BoardView's code should have gone into a robust controller class. It also recommended I improve coupling by creating a factory for checking different board types in BoardView so that I wouldn't have to make multiple modifications to it in the event that I added a new type of board. Restructuring the code so that there was a single class keeping track of game state, which managed ManualGame and AutomatedGame and could communicate well with BoardView, and which also overtook many of BoardView's functions, was a very thorny challenge and at this point in the project I was too tired to want to do it, and so I had Claude do the refactoring for me. The code works just as it did before, and the structure of the project is much improved in all the ways Claude recommended. I can't help but be a little upset, however, by how I no longer recognize a lot of this code as my own. It messed up some of the comments, too, adding to that feeling of alienation. I hope that in the future I can do a better job applying design principles the first time around so that I never have to give up my sense of ownership and pride in this way again.

Summary of the most impactful issues to discuss in your assignment:

1. Duplicated direction logic between `Board.validMovesExist()` and `AutomatedGame.getValidMoves()` — cohesion/modularity issue
2. Redundant `board` parameter in `AutomatedGame.makeRandomMove()` — coupling issue
3. `BoardView` managing its own `gameOver` state — encapsulation issue
4. `BoardView`'s `instanceof` chain for board types — coupling issue

Claude Chats:

- https://claude.ai/share/026263c5-962e-4914-9bde-ba05e250fd13 (Continuation of sprint 2's log)
- https://claude.ai/share/978b5ee1-d26f-4582-b625-9b5f57cb0ccb (Piecemeal help while implementing the ability to change board size and type)
- https://claude.ai/share/59c480e9-fa29-45be-8e96-fc414c5994f7 (Feedback on design principles and refactoring help)